C SC 367 – Fall 2026 – Midterm – Oct 23, 2026

INSTRUCTOR: G. Srivastava **STUDENT NUMBER:** 240955

TOTAL MARKS:

This is a closed book test. Write your answers on the question paper in the spaces provided. Use the reverse side for rough work. The total time allowed is 60 minutes. A handout on ARM instructions is provided.

Question 1. [6] State the addressing modes of the rightmost (bold) operands in the following ARM instructions. You do not need to give the addressing mode of the leftmost (destination) operand as it is always the same.

`MOV R4,R5`	Register Direct
`SUBS R4,R5,#7`	Immediate
`LDR R4,[R5]`	Register indirect
`LDR R4,[R5],#8`	Post-indexed immediate offset
`STR R4,[R5,R6,LSL #2]`	Pre-indexed scaled register offset with a logical shift left by 2
`STR R4,[R5,#8]!`	Pre-indexed immediate offset with writeback

Question 2. [10] State what the following ARM instructions do. Explain the behaviour in precise English or pseudo code and then after also give the final values in the registers if possible.

`SUBS R1,R2,#5`
`with R2=0xD0010`
`and R1=0xD004`
	R1 = 851,984 - 5; sets CPSR to 0010
	R1 == 0xD000B
	R2 == 0xD0010

`LDR R1,[R2,#8]`
`with R2=0xD0010`
`and R1=0xD004`
	R1 = valueFromAddress(R2 + 8);
	R1 == valueFromAddress(R2 + 8)
	R2 == 0xD0010

`LDR R1,[R2],#8`
`with R2=0xD0010`
`and R1=0xD004`
	R1 = valueFromAddress(R2);
	R2 += 8;
	R1 == value
	R2 == 0xD0018

`STR R1,[R2,R3,LSL #1] with R3=0xD004 and R2=0xD0000 and R1=0xC002`	Stores R1 to address R2 pre offset by R3 with a left bitshift of 1. R1 == 0xC002, R2 == 0xD0000, R3 == 0xD004

`STR R1,[R2,#8]!`
`with R2=0xD0010`
`and R1=0xD004`
	R2 += 8;
	Store R1 at address R2;
	R1 == 0xD004
	R2 == 0xD0018

Question 3. [12] Consider the diagram in Figure 1 describing the main elements of a possible CPU design with its internal datapath connections (as seen in the lectures). Use the diagram as a basis to explain the detailed steps in the Fetch/Decode/Execute cycles of the following 2 ARM instructions:

```
SUB R1,R2,#5
LDR R1,[R2,#8]!
```

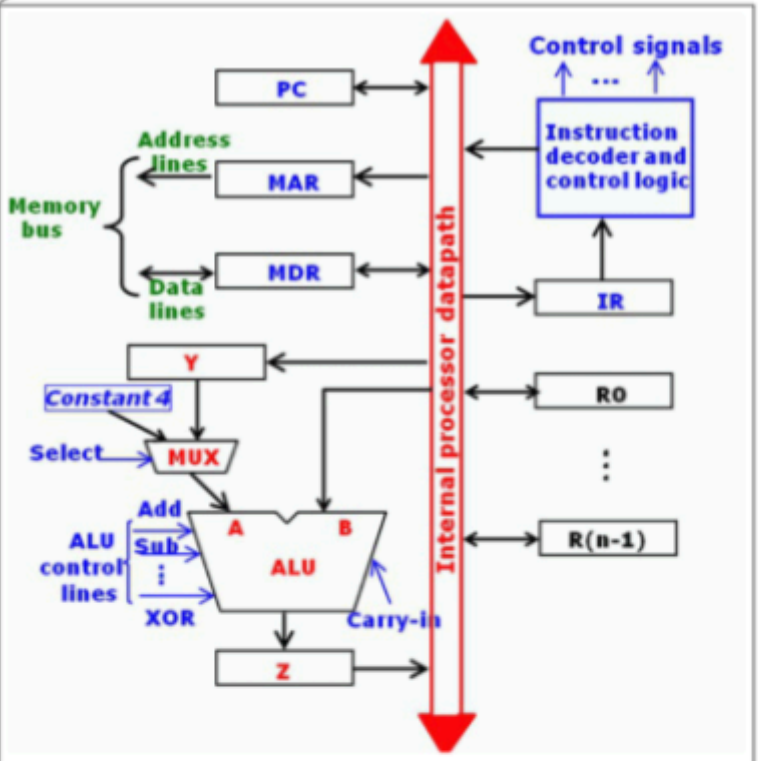


FIGURE 1. Possible internal organization of a processor

[4] FETCH PHASE (same for both)

1. PC = Mem address
2. MAR <- PC
3. Address is placed on address bus by MAR
4. CPU reads memory with a signal on the control bus
5. 32-bit instruction is returned on data bus to MDR
6. IR <- MDR
7. PC += 4

[4+4] DECODE AND EXECUTION PHASES

SUB R1,R2,#5

Decode:

1. Decoder recognizes SUB instruction

Execute:

1. ALU computes subtraction of 5 from R2
2. Output of ALU stored in R1

LDR R1,[R2,#8]!

Decode:

1. Decoder recognizes LDR instruction
2. Addressing mode is indirect and pre incrementing by 8 with writeback

Execute:

1. ALU computes the addition of 8 to R2
2. Output of ALU stored in R2
3. MAR <- R2
4. R2 then placed on address bus by MAR
5. CPU READs memory via control bus
6. 32-bit memory content is returned on data bus to MDR
7. R1 <- MDR

Question 5. [2] A computer system has a RAM containing 32K bytes, each of which needs its own distinct address. Moreover it has 6 peripherals and they each require 16 distinct addresses in order to interface properly.

(a) How many distinct addresses in total are necessary in this system? 2^5 + 2^6 + 2^15
(You do not need to compute a final number in decimal, just stating the calculation is sufficient)

(b) How many lines does an address bus for this system require, given that it must be able to carry all the needed values for the addresses? 16

Question 6. [8] The C language provides two different ways to process the elements of an array, as illustrated by the following two versions of a function for summing the elements of an array.

```
int Plus1( int *L, int N ) {
        int i, p;
    i = p = 0;
    while( i<N ) {
        p += L[i]; i++;
    }
    return p;
}
```

```
int Plus2( int *L, int N ) {
       int *LEnd, p;
    p = 0;
    LEnd = L+N;
     while( L < LEnd ) {
         p += *L; L++;
    }
    return p;
}
```

(a) [4] Assuming that `L`, `N`, `i` and `p` are held in registers R1, R2, R3 and R4 respectively, give the ARM assembly language code which corresponds to the two statements `p +=L[i]; i++;` in *Plus1*. Please note carefully that you must give the code ***only*** for the two statements, *not* for the whole loop/function.

```
ldr r5, [r1, r3, asl #4]      @ stores value of L[i]
add r4, r4, r5                @ adds L[i] to p

add r3, r3, #1                @ increments i
```

(b) [4] Assuming that `L`, `N`, `LEnd` and `p` are held in registers R1, R2, R3 and R4 respectively, give the ARM assembly language code which corresponds to the two statements `p+=*L; L++;` in *Plus2*.

```
ldr r5, [r1], #4              @ both stores the value of *L and increments L
add r4, r4, r5                @ adds L* to p
```

Question 8. [8] Write the ARM code segment to implement the loop structure given below in C. Make sure to add comments. Assume that X has been declared in the DATA section as 'X: .skip 4' `X=12;`

```
for (k=80;k>0;k--){
     X = X+5;
}
```

```
          MOV  r0 #12          @ stores starting value of X
          MOV  r1 #80          @ stores starting value of k

LOOP:     ADD  r0, r0, #5      @ start of loop; adds 5 to X
          SUBS r1, r1, #1      @ subtracts 1 from k; sets CPSR
          BGT  LOOP            @ returns to start of loop if k > 0

          LDR  r1 =X           @ overwrite r1 with address of X
          STR  r0, [r1]        @ store r0 to address of X

          SWI  0x11            @ end the program
```

Question 9. [5] Fill in the right column of the table with your answers.

Given 12 binary bits, the largest positive integer that can be represented using a 2's complement representation is:	2047
The hexadecimal value FE7 corresponds to the binary:	111111100111
The hexadecimal value FE7 viewed as an integer in a 2's complement context, corresponds to the decimal:	-000000011001 -25
Convert the decimal integer -37 to 8-bit binary in 2's complement	-00100101 -> 11011011
Convert the decimal integer -37 to hexadecimal in 2's complement	DB